

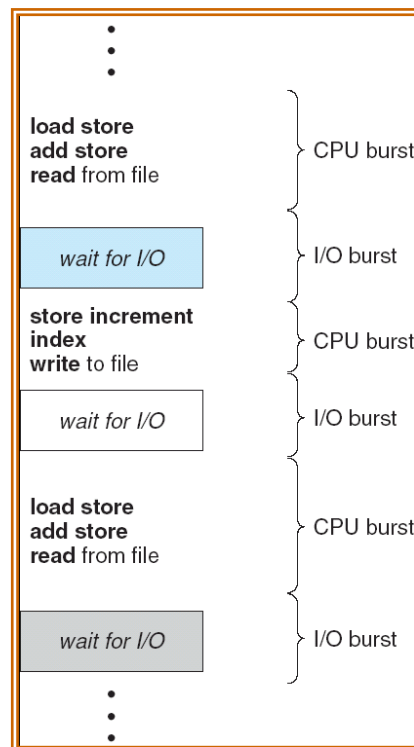
Chapter 5: CPU Scheduling

Operating System Concepts, 7th Edition — Silberschatz, Galvin, Gagne

1. Basic Concepts

Maximum CPU utilization is obtained through multiprogramming — keeping several processes in memory so the CPU always has something to execute.

CPU–I/O Burst Cycle: process execution consists of a cycle alternating between a CPU burst (computation) and an I/O burst (waiting for I/O). A typical process alternates: CPU burst → I/O burst → CPU burst → I/O burst → ... → final CPU burst, ending with a system request to terminate. Most CPU-burst-length distributions show a large number of short bursts and a small number of long bursts.



2. CPU Scheduler and Dispatcher

2.1 CPU Scheduler

The CPU scheduler selects, from among the processes in memory that are ready to execute, one process and allocates the CPU to it. CPU-scheduling decisions may take place when a process:

- 1. Switches from running to waiting state
- 2. Switches from running to ready state
- 3. Switches from waiting to ready state
- 4. Terminates

Scheduling under circumstances 1 and 4 is nonpreemptive (the process keeps the CPU until it voluntarily gives it up or finishes). All other scheduling is preemptive (the OS can forcibly take the CPU away).

2.2 Dispatcher

The dispatcher module gives control of the CPU to the process selected by the short-term scheduler. This involves:

- Switching context
- Switching to user mode
- Jumping to the proper location in the user program to restart it

Dispatch latency is the time it takes for the dispatcher to stop one process and start another running — this should be as small as possible.

3. Scheduling Criteria and Optimization Criteria

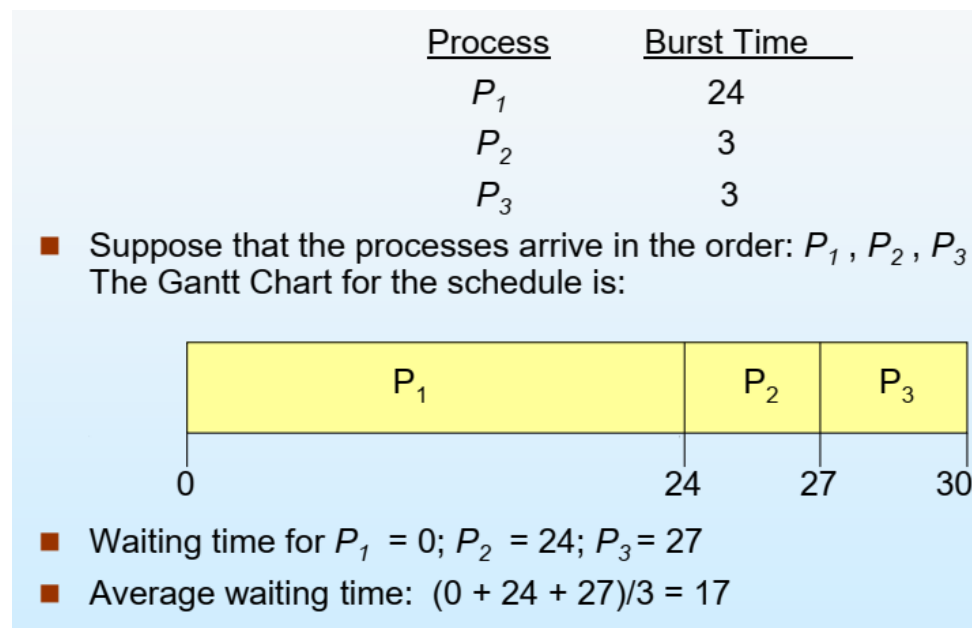
- **CPU utilization** — keep the CPU as busy as possible (maximize)
- **Throughput** — number of processes completed per unit time (maximize)
- **Turnaround time** — total time to execute a particular process, from submission to completion (minimize)
- **Waiting time** — total time a process has spent waiting in the ready queue (minimize)
- **Response time** — time from submission of a request until the first response is produced, not the full output, relevant in time-sharing systems (minimize)

4. Scheduling Algorithms

4.1 First-Come, First-Served (FCFS)

Processes are scheduled strictly in the order they arrive, using a simple FIFO queue. FCFS is nonpreemptive and easy to implement, but average waiting time can be quite long and highly dependent on arrival order.

Example: with burst times $P_1=24$, $P_2=3$, $P_3=3$, arriving in the order P_1, P_2, P_3 gives waiting times 0, 24, 27 (average 17). Arriving in the order P_2, P_3, P_1 instead gives waiting times 6, 0, 3 (average 3) — much better. This ordering sensitivity, where a short process gets stuck behind a long one, is called the convoy effect.



4.2 Shortest-Job-First (SJF)

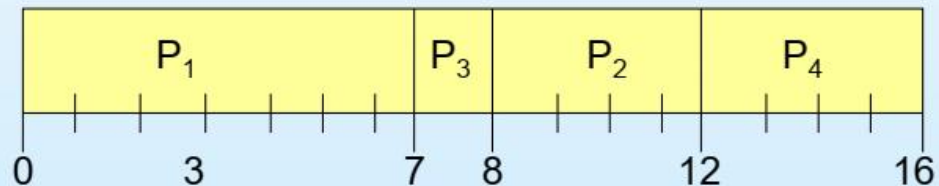
SJF associates with each process the length of its next CPU burst, and schedules the process with the shortest time first. SJF is provably optimal — it gives the minimum average waiting time for a given set of processes.

- **Nonpreemptive SJF** — once the CPU is given to a process, it cannot be preempted until it completes its current CPU burst
- **Preemptive SJF (Shortest-Remaining-Time-First, SRTF)** — if a new process arrives with a CPU burst shorter than the remaining time of the currently executing process, the CPU is preempted in its favour

Example of Non-Preemptive SJF

Process	Arrival Time	Burst Time
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

■ SJF (non-preemptive)

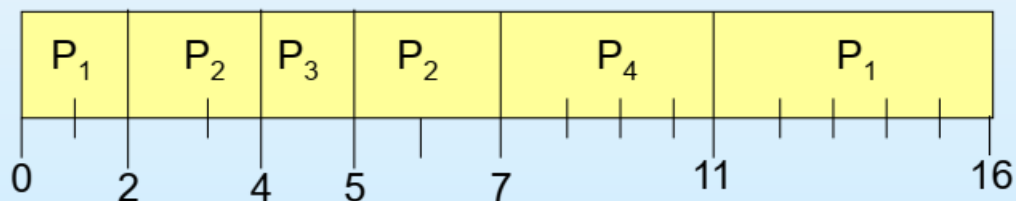


■ Average waiting time = $(0 + 6 + 3 + 7)/4 = 4$

Example of Preemptive SJF

Process	Arrival Time	Burst Time
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

■ SJF (preemptive)



■ Average waiting time = $(9 + 1 + 0 + 2)/4 = 3$

Since actual burst length is not known in advance, it is typically predicted using exponential averaging over past bursts: $\tau_{n+1} = \alpha \cdot t_n + (1-\alpha) \cdot \tau_n$, where t_n is the actual length of the n th burst, τ_n is the previous prediction, and $0 \leq \alpha \leq 1$. When $\alpha = 0$, recent history does not count ($\tau_{n+1} = \tau_n$); when $\alpha = 1$, only the most recent burst counts ($\tau_{n+1} = t_n$). Expanding the recurrence shows that each successive past term is weighted less than its predecessor, since both α and $(1-\alpha)$ are ≤ 1 .

4.3 Priority Scheduling

A priority number (integer) is associated with each process, and the CPU is allocated to the process with the highest priority (by convention, the smallest integer is the highest priority). Priority scheduling can be preemptive or nonpreemptive. SJF is in fact a special case of priority scheduling, where the priority is the (predicted) length of the next CPU burst.

A key problem with priority scheduling is starvation — low-priority processes may wait indefinitely and never execute. The standard solution is aging: gradually increasing the priority of a process the longer it waits.

4.4 Round Robin (RR)

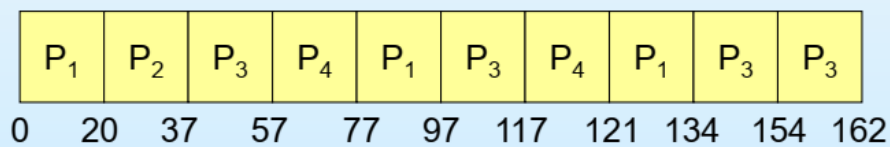
Each process is given a small unit of CPU time called a time quantum (typically 10–100 ms). Once this time elapses, the process is preempted and placed at the end of the ready queue. If there are n processes in the ready queue and the time quantum is q , each process gets $1/n$ of the CPU time in chunks of at most q time units, and no process waits more than $(n-1)q$ time units.

Performance depends heavily on the size of q : if q is very large, RR behaves like FCFS; if q is very small, the overhead from frequent context switches becomes too high relative to useful work, so q should be large compared to context-switch time. RR generally has higher average turnaround time than SJF but gives much better response time, which is important for interactive/time-shared systems.

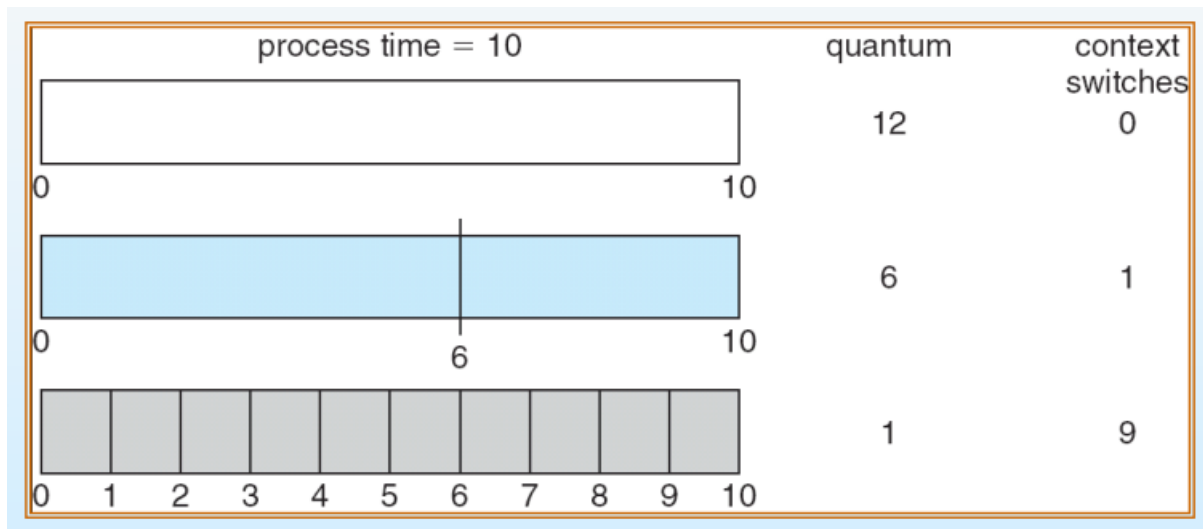
Example of RR with Time Quantum = 20

Process	Burst Time
P_1	53
P_2	17
P_3	68
P_4	24

■ The Gantt chart is:



■ Typically, higher average turnaround than SJF, but better *response*

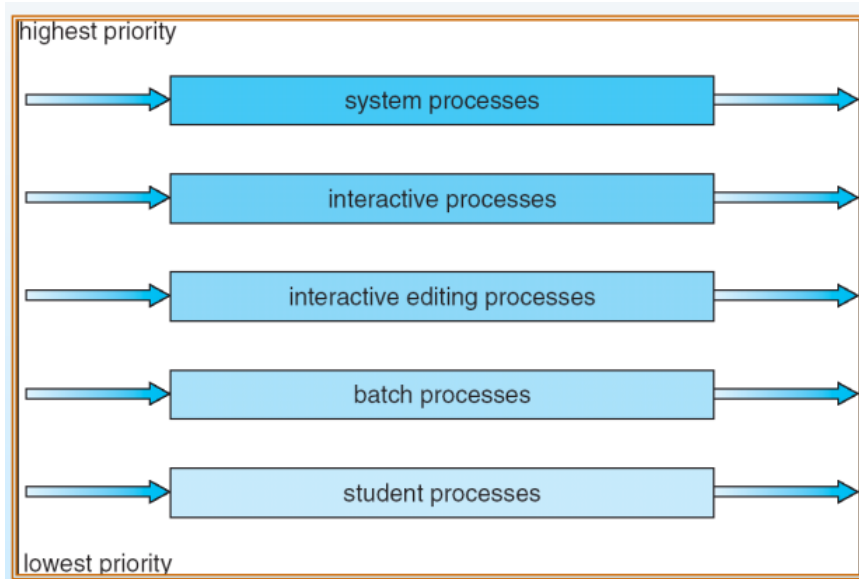


4.5 Multilevel Queue

The ready queue is partitioned into several separate queues, e.g., foreground (interactive) and background (batch), based on some process property such as memory size, priority, or process type. Each queue has its own scheduling algorithm — e.g., Round Robin for foreground and FCFS for background.

Scheduling must also be done between the queues themselves, typically either:

- Fixed-priority scheduling — serve all processes from the higher-priority queue first, then move to the next; this risks starvation of lower-priority queues
- Time slicing — each queue gets a certain fraction of CPU time to schedule among its own processes (e.g., 80% to foreground RR, 20% to background FCFS)

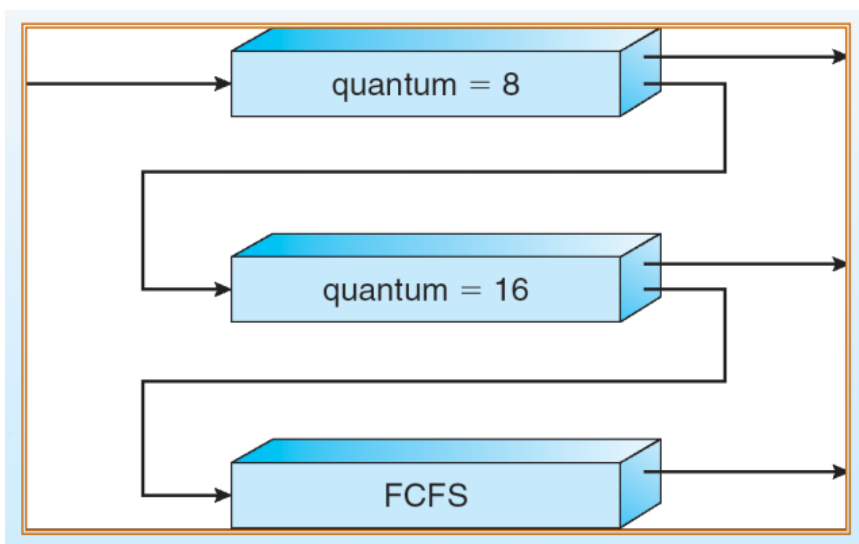


4.6 Multilevel Feedback Queue

This generalizes the multilevel queue by allowing a process to move between queues; aging can be implemented this way, since a process that waits too long in a lower-priority queue can be promoted. A multilevel-feedback-queue scheduler is defined by:

- The number of queues
- The scheduling algorithm used for each queue
- The method used to determine when to upgrade a process to a higher-priority queue
- The method used to determine when to demote a process to a lower-priority queue
- The method used to determine which queue a process enters when it needs service

Example (three queues): Q0 uses RR with time quantum 8 ms; Q1 uses RR with time quantum 16 ms; Q2 uses FCFS. A new job enters Q0; if it does not finish within 8 ms, it moves to Q1; if it still does not finish after 16 more ms there, it is preempted and moved to Q2, where it runs to completion FCFS.



5. Multiple-Processor Scheduling

CPU scheduling becomes more complex when multiple CPUs are available.

- Homogeneous processors — within a multiprocessor system, any process can run on any available processor
- Load sharing — the workload is distributed across all processors, generally via a common ready queue
- Asymmetric multiprocessing — only one processor (the master) accesses the system data structures and makes all scheduling decisions, which alleviates the need for data sharing among processors

6. Real-Time Scheduling

- Hard real-time systems — a critical task must be guaranteed to complete within a specified amount of time; missing a deadline is a system failure
- Soft real-time computing — critical processes simply receive priority over other, less critical processes, but no hard guarantee is given

Multiple Choice Questions (10)

1. Maximum CPU utilization is best achieved through:

- a) Single programming
- b) Multiprogramming
- c) Batch processing only
- d) Disabling interrupts

Answer: (b) Multiprogramming

2. Scheduling that occurs only when a process switches from running to waiting, or terminates, is called:

- a) Preemptive scheduling
- b) Nonpreemptive scheduling
- c) Real-time scheduling
- d) Multilevel scheduling

Answer: (b) Nonpreemptive scheduling

3. The time it takes for the dispatcher to stop one process and start another running is called:

- a) Turnaround time
- b) Waiting time
- c) Dispatch latency
- d) Response time

Answer: (c) Dispatch latency

4. Which scheduling criterion is most relevant to a time-sharing/interactive environment, measuring time until the first response is produced?

- a) Throughput
- b) Response time
- c) CPU utilization
- d) Turnaround time

Answer: (b) Response time

5. The scheduling algorithm that is provably optimal in giving the minimum average waiting time for a given set of processes is:

- a) FCFS
- b) Round Robin
- c) Shortest-Job-First (SJF)
- d) Multilevel Queue

Answer: (c) Shortest-Job-First (SJF)

6. In FCFS scheduling, a short process getting stuck behind a long process in the ready queue is known as:

- a) Starvation
- b) Aging
- c) The convoy effect
- d) Priority inversion

Answer: (c) The convoy effect

7. The preemptive version of Shortest-Job-First scheduling is also known as:

- a) Shortest-Remaining-Time-First (SRTF)
- b) Round Robin
- c) Multilevel Feedback Queue
- d) FCFS

Answer: (a) Shortest-Remaining-Time-First (SRTF)

8. In priority scheduling, the technique of gradually increasing the priority of processes that wait a long time, to prevent starvation, is called:

- a) Load sharing
- b) Aging
- c) Time slicing
- d) Dispatching

Answer: (b) Aging

9. In Round Robin scheduling, if the time quantum is set very large, the algorithm's behavior approaches that of:

- a) SJF
- b) Priority scheduling
- c) FCFS
- d) Multilevel feedback queue

Answer: (c) FCFS

10. A multiprocessor scheduling approach where only one processor accesses the system data structures, removing the need for data sharing among processors, is called:

- a) Load sharing
- b) Symmetric multiprocessing
- c) Asymmetric multiprocessing
- d) Homogeneous processing

Answer: (c) Asymmetric multiprocessing

Five-Mark Questions (5)

1. Explain the CPU–I/O burst cycle and describe how CPU-scheduling decisions relate to the different points at which a process can be scheduled (running→waiting, running→ready, waiting→ready, termination). Distinguish preemptive from nonpreemptive scheduling in this context.

2. Define the five scheduling criteria (CPU utilization, throughput, turnaround time, waiting time, response time) and state the optimization goal (max/min) for each. Explain why response time is emphasized in interactive systems while turnaround time matters more for batch systems.

3. Consider the 3 processes, P1, P2 and P3 are shown in the table

Process	Arrival time	Burst Time
P1	0	5
P2	1	7
P3	3	4

Draw the Gantt Chart and calculate the average waiting time and avg turnaround time using FCFS (First-Come First-Serve), Preemptive SJF (Shortest Job First) and Nonpreemptive SJF algorithms.

4. Consider the 4 processes, P1, P2, P3 and P4 are shown in the table

Process	Arrival time	CPU Time
P1	0	13
P2	2	16
P3	3	10
P4	5	8

Draw the Gantt Chart and calculate the average waiting time and average turnaround time using RR (Round Robin) algorithm with time quantum (q)= 3.

5. Describe multilevel queue scheduling and multilevel feedback queue scheduling. Explain the parameters that define a multilevel-feedback-queue scheduler, and illustrate with an example of how a process can move between queues.